Solution: Group Anagrams

Problem Statement

Given a list of strings, the task is to group the anagrams together.

An anagram is a word or phrase formed by rearranging the letters of another, such as "cinema", formed from "iceman".

Example Generation

Example 1:

- **Input**: `["dog", "god", "hello"]`
- **Output**: `[["dog", "god"], ["hello"]]`
- **Justification**: "dog" and "god" are anagrams, so they are grouped together. "hello" does not have any anagrams in the list, so it is in its own group.

Example 2:

- **Input**: `["listen", "silent", "enlist"]`
- **Output**: `[["listen", "silent", "enlist"]]`
- **Justification**: All three words are anagrams of each other, so they are grouped together.

Example 3:

- **Input**: `["abc", "cab", "bca", "xyz", "zxy"]`
- **Output**: `[["abc", "cab", "bca"], ["xyz", "zxy"]]`
- **Justification**: "abc", "cab", and "bca" are anagrams, as are "xyz" and "zxy".

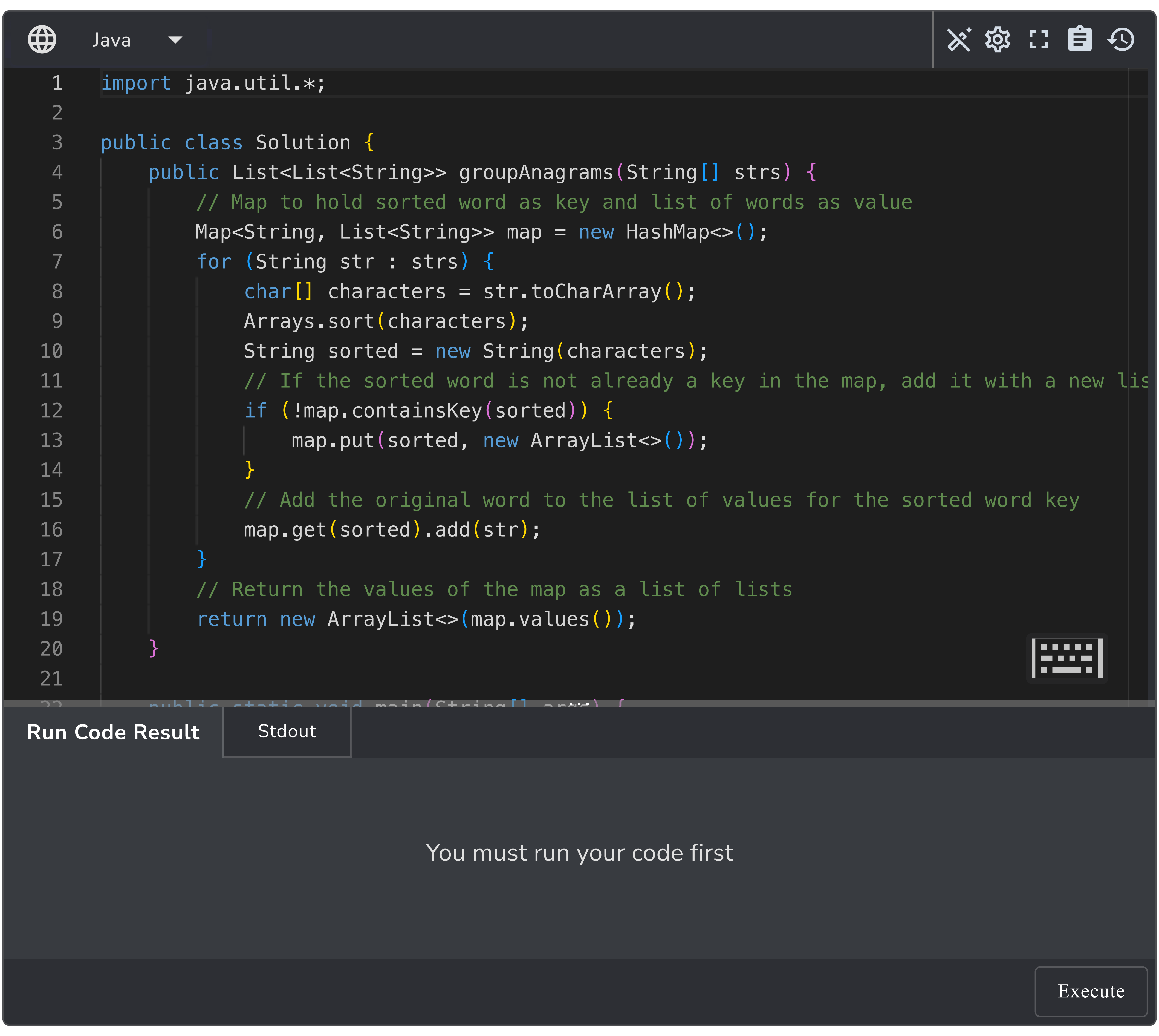
Solution

- **Sorting Approach**:
  - For each word in the input list:
    - Sort the letters of the word.
    - Use the sorted word as a key in a hash map, and add the original word to the list of values for that key.
  - The hash map values will be the groups of anagrams.
- **Why This Will Work**:
  - Anagrams will always result in the same sorted word, so they will be grouped together in the hash map.

Algorithm Walkthrough

- Given the input `["abc", "cab", "bca", "xyz", "zxy"]`
- For "abc":
  - Sorted word is "abc".
  - Add "abc" to the hash map with key "abc".
- For "cab":
  - Sorted word is "abc".
  - Add "cab" to the list in the hash map with key "abc".
- Continue this process for all words.
- The hash map values are the groups of anagrams.

Code



```java
import java.util.*;

public class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        // Map to hold sorted word as key and list of words as value
        Map<String, List<String>> map = new HashMap<>();
        for (String str : strs) {
            char[] characters = str.toCharArray();
            Arrays.sort(characters);
            String sorted = new String(characters);
            // If the sorted word is not already a key in the map, add it with a new lis
            if (!map.containsKey(sorted)) {
                map.put(sorted, new ArrayList<>());
            }
            // Add the original word to the list of values for the sorted word key
            map.get(sorted).add(str);
        }
        // Return the values of the map as a list of lists
        return new ArrayList<>(map.values());
    }

```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n*k*log(k)), where n is the number of strings, and k is the maximum length of a string in strs. This is because each of the n strings is sorted in O(k*log(k)) time.
- **Space Complexity**: O(n*k), where n is the number of strings, and k is the maximum length of a string in strs. This space is used for the output data structure and the hash map.

← Previous
Group Anagrams (Medium)

Next →
Decode String (Medium)

☐ Mark as Completed